

Human-AI Teaming Framework

Making Semantic Governance Trustworthy at Scale

Jeffrey Wallk

Architecting Knowledge & Trusted AI

February 2026

Introduction

We've built an impressive semantic governance architecture with federated RAG 2.0, AI circuit breakers, and comprehensive service portfolios. The technical infrastructure is world-class, grounded in trustworthy engineering principles that ensure our AI systems are reliable, explainable, and safe. But there's a dimension of trustworthiness we haven't fully addressed yet - the human dimension.

Our trustworthy engineering approach has focused on making AI systems technically sound: preventing hallucinations through circuit breakers, ensuring semantic consistency through federated knowledge graphs, maintaining explainability through structured reasoning. These are essential foundations. But trustworthiness doesn't end with the technology. It extends to the people who supervise it, validate its outputs, and intervene when needed.

Research across industries - from aviation to healthcare to industrial control - has documented a consistent pattern when sophisticated automation meets human operators. The better the automation gets, the worse human supervisors perform when that automation needs oversight. This isn't a failure of the people or the technology individually. It's a failure to design the human-AI interaction system as carefully as we design the AI itself.

Air France 447 crashed because pilots couldn't take over when the autopilot disengaged. Healthcare systems saw a 30% increase in preventable deaths after implementing alert systems that fatigued medical staff. Three Mile Island partially melted down because operators had lost the skills to manually control systems they'd been monitoring for years. Knight Capital lost \$440 million in 45 minutes because traders over-trusted algorithmic trading systems.

Our semantic governance system could follow the same trajectory. Knowledge engineers who become complacent as AI accuracy improves, who start rubber-stamping validations, whose manual mapping skills atrophy, and who can't effectively intervene when the AI fails. This framework prevents that outcome by extending our trustworthy engineering principles to encompass the entire human-AI system.

Extending Trustworthy Engineering to Human-AI Teams

Our existing trustworthy engineering framework establishes rigorous standards for AI system behavior. The circuit breaker framework prevents unsafe actions through multi-layer validation. The federated architecture ensures semantic consistency across domains. The vector database enables contextual grounding that reduces hallucinations. These technical safeguards make the AI itself trustworthy.

The Human-AI Teaming Framework extends these same principles to the people who work with the AI. Just as we instrument the AI to detect drift, hallucinations, and unsafe states, we instrument human performance to detect complacency, fatigue, and miscalibrated trust. Just as we have circuit breakers that halt AI actions when safety thresholds are crossed, we have interventions that halt human work when performance degrades. Just as we maintain the AI's capability through continuous learning, we maintain human capability through competency exercises.

This creates a complete trustworthy engineering system where both the AI and the humans are monitored, supported, and maintained at high performance levels. The AI doesn't degrade because we have drift detection and retraining. The humans don't degrade because we have fatigue detection and competency maintenance. The combination is trustworthy in a way that neither component alone could be.

The Trustworthy Human-AI System

A trustworthy AI system needs three things: reliable AI, capable humans, and well-designed interaction between them. We've built the first piece with our semantic governance architecture. This framework builds the second and third pieces.

Reliable AI means the technical components work as designed - the circuit breakers trip appropriately, the knowledge graphs maintain semantic consistency, the vector databases provide relevant context. We've accomplished this through rigorous engineering, comprehensive testing, and continuous monitoring. Our AI systems meet the standards for safety, explainability, and reliability.

Capable humans means knowledge engineers maintain the skills, judgment, and cognitive capacity to effectively supervise the AI. This requires active design. We can't assume that people who monitor automated systems will naturally stay sharp. They need feedback systems that calibrate trust, workload management that prevents fatigue, and competency programs that preserve skills. These aren't nice-to-have features - they're essential components of a trustworthy system.

Well-designed interaction means the interface between humans and AI makes good decisions easy and bad decisions hard. When the AI presents a high-confidence suggestion that's actually wrong, the interface should make that detectable. When an engineer is fatigued and likely to make errors, the system should recognize it and intervene. When pressure to meet deadlines starts compromising quality, the organization should see it and respond. The interaction layer is where human capability and AI capability combine to produce system-level trustworthiness.

Complementing the Circuit Breaker Framework

The AI Circuit Breaker Framework establishes technical boundaries: confidence thresholds, semantic drift limits, hallucination detection, multi-layer validation logic. When the AI crosses these boundaries, the circuit breaker trips and requires human intervention. This is essential for preventing the AI from taking unsafe actions.

But what happens when the circuit breaker trips? A human needs to review the situation, understand what went wrong, make a judgment about how to proceed, and potentially take manual action. If that human is fatigued, complacent, or has lost their manual skills, the circuit breaker hasn't actually made the system safe - it's just shifted the failure point.

The Human-AI Teaming Framework complements the circuit breaker by ensuring humans can effectively respond when it trips. We monitor whether engineers are in a cognitive state to handle complex interventions. We maintain their skills so they can actually perform manual mappings when needed. We calibrate their trust so they don't just override the circuit breaker to clear the queue. We detect organizational pressure that might lead them to accept marginal decisions.

Together, the two frameworks create defense in depth. The AI circuit breaker prevents technical failures. The human performance monitoring prevents supervisory failures. The combination is robust against both AI errors and human errors, which is what trustworthiness requires.

Enhancing the Federated Architecture

The federated semantic governance architecture distributes knowledge across domain-specific agents while maintaining global consistency. This design prevents any single point of failure and allows specialized expertise in each domain. It's a sophisticated technical architecture that embodies trustworthy engineering principles.

The human interaction with this federated system presents challenges that pure technical design can't solve. When multiple agents are active, how does an engineer know which domains are represented in their query results? When subdomain filters are applied, how do they recognize what might be missing? When the system returns results from Search and Discover but not Federation, what indicates the incompleteness?

The Human-AI Teaming Framework adds mode awareness and context transparency to the federated architecture. Engineers see which agents are active, what filters are applied, what context is being used. When they make a validation decision, they see the full scope of what the AI considered and what it didn't. This prevents mode confusion - the state where people don't understand what the system is actually doing and make decisions based on incomplete information.

This extends the trustworthiness of federation from the technical layer to the interaction layer. The architecture is trustworthy because knowledge is properly distributed and semantically consistent. The interaction is trustworthy because engineers understand what they're working with and can make informed decisions.

Supporting the Knowledge Graph Foundation

The knowledge graph provides the semantic foundation for the entire system - relationships, taxonomies, ontologies, and mappings that create shared meaning across the enterprise. The vector database and context engine make this foundation computationally accessible through semantic search and contextual grounding. This combination enables AI agents to reason about data with proper semantic context.

Maintaining this foundation requires human judgment. When should two concepts be mapped? What level of confidence is sufficient for automatic acceptance? How do we handle ambiguous cases where multiple valid interpretations exist? These aren't questions the AI can answer alone - they require human expertise in the domain, understanding of downstream use cases, and judgment about acceptable tradeoffs.

The Human-AI Teaming Framework ensures that human judgment remains sound as the foundation grows. When engineers validate mappings, they receive explainability that shows why the AI suggested what it did - not just a confidence score, but the actual evidence, potential concerns, and historical calibration data. This transforms validation from 'accept or reject' to 'understand and decide,' which is what semantic governance actually requires.

We also prevent the knowledge graph from accumulating errors through systematic oversight failures. Trust calibration ensures engineers don't over-accept suggestions in certain confidence ranges. Competency maintenance ensures they can still create quality mappings manually when needed. Production pressure monitoring ensures organizational demands don't compromise the integrity of the semantic foundation.

The result is a knowledge graph that's trustworthy not just because it's well-engineered, but because every human decision that shaped it was made by someone in a cognitive state to make good decisions, with access to full context, and without systematic biases toward over-acceptance.

The Automation Paradox

Here's what happens in practice. Right now, our AI suggests mappings with around 65% accuracy. Knowledge engineers review each suggestion carefully because they know the AI makes mistakes. They're engaged, building domain expertise, maintaining their skills. The system works at about 95% reliability.

Fast forward a year. The AI is now hitting 92% accuracy - much better. But something unexpected has happened. Engineers have started trusting the AI recommendations without really checking them. Why wouldn't they? The AI is right most of the time. Their reviews have become perfunctory. When they see a high confidence score, they accept it. They're not building domain knowledge anymore because they don't need to think deeply about each decision. Their manual mapping skills have gotten rusty from disuse.

Now when the AI fails - and it still fails 8% of the time - the engineers don't catch it. The system reliability has dropped to 85%. We improved the AI from 65% to 92%, and the overall system got worse. That's the paradox.

This isn't speculation. It's been measured and documented across industries. The pattern is so consistent that you can predict it. Better automation leads to worse human performance, which leads to worse overall system performance, which eventually leads to catastrophic failure when the automation fails in an unexpected way and humans can't take over.

From a trustworthy engineering perspective, this paradox represents a fundamental design flaw. We've optimized one component (the AI) without considering its impact on the other component (the human) or the interaction between them. A truly trustworthy system optimizes the whole, not the parts in isolation.

Human Performance Challenges in AI Systems

The architecture faces several human performance challenges that trustworthy engineering must address:

Automation Complacency

When engineers see a confidence score of 0.85, they're going to accept it without much thought unless we actively prevent this. The system needs to detect when validations are becoming perfunctory, measure if engineers are actually checking or just clicking through, and intervene before bad mappings propagate through 30,000+ data assets. This isn't a failure of the engineers - it's a predictable human response to reliable automation that we need to design around.

Alert Fatigue

Our conflict resolution system flags items for manual review. That's good trustworthy engineering - don't auto-resolve conflicts. But what happens when an engineer gets 100 flags in a day? After a while, they stop reading them carefully. They start auto-accepting to clear the queue. Without managing this cognitive load, our careful conflict detection becomes ineffective because the human component degrades under volume.

Trust Mis-calibration

An engineer sees a confidence score of 0.85 and thinks that means the AI is probably right. But what if the AI's actual accuracy at 0.85 is only 62%? The engineer is over-trusting the system and accepting bad mappings. Or what if accuracy is 91% but the engineer treats it like it's 70% and wastes time on unnecessary validation? Neither case serves trustworthiness. We need trust levels to match reality.

Skill Degradation

After six months of the AI handling most mappings, engineers lose their ability to create mappings manually. When the AI fails or encounters something unusual, they can't effectively intervene. They've become dependent on a system they're supposed to be supervising. From a trustworthy engineering standpoint, this represents a dangerous single point of failure - if the AI fails, we have no fallback because human capability has atrophied.

Cognitive Overload

Someone who's been making validation decisions for six straight hours isn't performing at the same level as someone who just started their shift. We've measured this - accuracy drops 40% or more. Our system treats engineers like machines that maintain constant performance regardless of working conditions. That's not trustworthy engineering - that's ignoring a critical variable in system performance.

Mode Confusion

We have multiple domain agents - Search and Discover, Federation, Orchestration. An engineer makes a query and gets results, but they don't actually know which agents were active, what context was applied, or what might be missing. The system returns incomplete results and the user has no indication. Decisions get made on partial information. This violates the transparency principle that's central to trustworthy AI.

Production Pressure

When management pushes to complete 100 mappings per day, quality suffers. Engineers start rushing, overriding safety alerts to meet targets, making decisions faster than they should. These patterns are detectable - decision speed increasing, accuracy declining, excessive alert overrides. Without monitoring and protecting engineers' authority to halt work when quality is at risk, organizational pressure gradually erodes safety culture until something fails catastrophically.

The Framework

The Human-AI Teaming Framework extends trustworthy engineering principles to human performance through seven integrated capabilities. These aren't theoretical add-ons. They're practical systems that track human performance in real-time, provide feedback that calibrates trust, manage cognitive load, maintain competency, and protect organizational safety culture.

Real-Time Performance Monitoring

Every validation decision gets tracked with full context: what the decision was, how long it took, what the engineer's cognitive state was at the time, whether the decision was correct. We calculate rolling accuracy over the last 20 decisions, monitor how decision speed compares to baseline, track how long the session has been running, measure time since last break.

When we detect problems, we alert immediately. If accuracy drops more than 15% below baseline, the engineer gets a warning to review recent decisions and consider taking a break. If decision speed is more than 2.5 times faster than normal, we flag it as rushing and recommend slowing down. When fatigue score crosses 0.7 on a 0-1 scale, we enforce a mandatory 15-minute break. If a session runs longer than six hours, we require it to end.

The fatigue score itself combines multiple factors. Session duration contributes 30% - six hours of continuous work gives you the maximum score of 1.0. Time since last break contributes another 30% - go 90 minutes without a break and you're at 1.0 for this factor. Accuracy degradation contributes 20% - if you've dropped 20% from your baseline, that's 1.0. Decision speed increase contributes the final 20% - if you're making decisions twice as fast as your baseline, that's 1.0. The weighted sum tells us how fatigued someone is, and we adjust work assignment and break enforcement accordingly.

This mirrors how we monitor AI systems - tracking drift, detecting anomalies, triggering interventions when thresholds are crossed. We're applying the same engineering discipline to human performance that we apply to AI performance.

Trust Calibration

For each confidence bucket - say 0.80 to 0.85 - we track two things: what percentage of suggestions the engineer accepts, and what percentage are actually correct. If an engineer accepts 94% of suggestions in this range but only 71% are actually correct, they're over-trusting by 23%. That's dangerous. They're systematically accepting bad mappings.

We classify this into states. If the gap between acceptance rate and actual accuracy is less than 5%, you're well-calibrated - no action needed. Between 5% and 15% is slight mis-calibration - we

give you awareness but no urgent warnings. Above 15% over-trust means you're accepting significantly more than you should - we warn you before each decision in that range. Under-trust of more than 15% means you're being too conservative - we let you know the AI is more reliable than you think.

The warnings appear in context. Before an engineer makes a decision on a mapping with 0.82 confidence, the system checks if they historically over-trust at that level. If they do, they see a message: You tend to over-trust AI suggestions in the 0.85 confidence range. You accept 94% but actual accuracy is 71%. Please review carefully. It's personalized, specific, and actionable.

This is explainability applied to human behavior, using the same principles we apply to AI behavior. Just as we explain why the AI made a suggestion, we explain why a human's trust might be mis-calibrated and provide corrective feedback.

Confidence Explainability

A confidence score of 0.82 doesn't mean much by itself. What we provide is a breakdown of where that score comes from. String similarity might contribute 25% and score 0.91 because there's an exact match on field name. Semantic similarity contributes 30% and scores 0.79 based on vector embeddings. Structural alignment contributes 20% and scores 0.88 because eight out of nine properties align. Historical precedent contributes 15% and scores 0.92 because there are 15 similar mappings that were validated with 92% accuracy. Domain usage contributes 10% and scores 0.75 because the schema was last modified two days ago.

We also show what the score means in context. If 0.82 is in the top 25% of scores for this domain pair, that's meaningful. If it's median, that's different. We compare it to historical accuracy - if suggestions at 0.82 confidence have actually been correct 71% of the time over the past three months, the AI is overconfident by 11% and we tell the engineer to treat it like 70% confidence instead.

Then there's evidence and concerns. Evidence might include exact string matches, known synonyms, similar validated mappings, data type alignment, usage pattern matches. Concerns might include test schemas, stale schemas that haven't been modified in a year, missing foreign keys, multiple candidate schemas creating ambiguity, data type mismatches, or confidence drift where historical accuracy doesn't match the score.

Based on all this, we generate a validation checklist. Basic validation always includes verifying production schemas, confirming data type compatibility, checking that concept meaning aligns with schema purpose. If there are concerns, we add specific items: if there are multiple candidates, the checklist says to review the alternatives. If there's a data type mismatch, it says to resolve it or confirm transformation exists. If we detect confidence drift, it says extra scrutiny is needed because AI confidence may be inflated.

This transforms the interaction from 'trust the AI' to 'understand the AI's reasoning and make an informed decision.' That's the essence of trustworthy human-AI teaming.

Cognitive Workload Management

Task assignment isn't random. We assess each engineer's current cognitive state by looking at session duration, time since last break, number of decisions made, recent accuracy trends, and recent decision speed. These combine into a cognitive capacity score from 0 to 1, where 1 is fresh and 0 is exhausted. If someone needs a break, we don't assign them anything - we tell them to take 15 minutes. If all available tasks exceed their current capacity, same thing - no assignment, suggestion to take a break.

For suitable tasks, we prioritize intelligently. If someone is fresh with capacity above 0.8, we prefer challenging tasks with complexity above 0.7 to maintain engagement. If they're fatigued below 0.5, we prefer simple tasks below 0.4 to prevent errors. In between, we match task complexity to capacity. We also add variety - if a task type hasn't appeared in the last five tasks, it gets a bonus to prevent monotony. Business priority and deadline urgency factor in too, but cognitive matching is central.

This is load balancing applied to human cognitive resources, using the same principles we apply to computational resources. We monitor utilization, detect overload conditions, and distribute work to maintain optimal performance.

Competency Maintenance

Every 14 days, engineers do a manual mapping exercise without AI assistance. The system selects an unmapped concept of moderate difficulty, provides candidate schemas, and asks the engineer to create the mapping, document their reasoning, and assign a confidence score. Time limit is 30 minutes.

When they're done, we evaluate it. We compare their answer to what the AI would have suggested and to expert validation if available. We score reasoning quality by looking for evidence mentioned, alternatives considered, concerns or caveats noted, and specificity of documentation. If reasoning is minimal, they get low marks. If it's thorough and considers multiple angles, high marks.

The overall score combines correctness (60% weight) and reasoning quality (40% weight). We compare this to their baseline. If they score 15% or more below baseline, we flag skill degradation, recommend training, and schedule a follow-up assessment in seven days. Knowledge quizzes every 30 days and conflict resolution simulations every 21 days round out the program.

This is continuous validation applied to human capability, paralleling how we continuously validate AI models through testing and retraining. We don't assume capability persists - we measure it and maintain it actively.

Production Pressure Detection

We watch for patterns that indicate pressure is compromising quality. Individual indicators include rushing - making decisions 2.5 times faster than baseline. Accuracy decline of more than 10%. Excessive alert overrides above 20%. Burnout risk from three or more consecutive sessions over six hours. Off-hours work in more than 30% of sessions.

At the team level, we look for systemic issues. If more than 50% of the team shows rushing behavior, that's a workload problem, not individual performance. We calculate a pressure score from 0 to 1 based on severity-weighted indicators. Above 0.7 is critical and triggers management alerts with specific interventions: halt non-critical work, reduce daily targets by 30-50%, enforce mandatory time off for burned-out engineers, conduct emergency staffing assessment.

Management gets a dashboard showing team pressure score, pressure level classification, list of affected engineers with individual scores, breakdown by severity, recommended interventions with required authority level, historical trends, and a flag for whether immediate action is needed. The interventions are specific - not just 'there's a problem' but 'here's exactly what to do and who needs to authorize it.'

This is anomaly detection and circuit breaking applied to organizational behavior. Just as we detect when AI systems are behaving unsafely, we detect when organizational dynamics are pushing the system toward unsafe states.

Enhanced Interfaces

The mapping validation interface brings everything together. When an engineer opens a validation task, they see a trust warning banner if they have a history of mis-calibration at this confidence level. The confidence score displays as a colored badge with category label and percentile context. If historical calibration shows overconfidence, there's a warning. If the engineer has personal history of over-trusting at this level, another warning.

They can expand the confidence breakdown to see component scores. Evidence appears with strength indicators. Concerns are highlighted with severity levels. The validation checklist is interactive with progress tracking - critical items must be completed before the accept button enables. There's a reasoning field where engineers can document their decision, which feeds into quality analysis.

Tom Sawyer Workbench gets enhanced dashboards. The main view shows human performance metrics alongside technical metrics - current fatigue level, accuracy trend, trust calibration status. The pending work panel estimates review time and compares it to available time, with workload recommendations based on cognitive state. When they're looking at a specific mapping, they see the confidence analysis, graph visualization with impact analysis, evidence and concerns, and the validation checklist all in one place.

These interfaces embody the transparency principle from trustworthy engineering. The system's state is visible. The reasoning is explainable. The context is clear. This enables informed human judgment, which is the foundation of trustworthy supervision.

How It Integrates

This framework doesn't replace anything. It adds a parallel layer that extends trustworthy engineering to the complete human-AI system.

The Semantic Governance Architecture gets human performance monitoring, trust calibration, and workload management. The AI Circuit Breaker Framework gets human takeover protocols, mode awareness, and competency maintenance. The Vector DB and Context Engine get confidence explainability, evidence presentation, and concern identification. The Service Portfolio gets human performance services, competency tracking, and pressure monitoring. The Operating Model gets human-centered workflows, cognitive support, and alert fatigue management. Tom Sawyer Workbench gets human metrics dashboards, trust calibration views, and workload indicators.

The data flow works like this. An engineer starts a session. The Cognitive Workload Manager assesses their state and assigns an appropriate task. When they open the task, the interface requests confidence explanation and checks for any trust calibration warnings. The engineer makes a decision. Performance monitoring records it with full context and timing. If performance issues are detected, alerts fire. Trust calibration metrics update. If the engineer is fatigued, the system enforces a break. If pressure indicators appear, monitoring analyzes and may alert management.

Everything flows through a central Performance Database that stores decisions, sessions, trust events, and competency assessments. Services query this database to calculate metrics, detect patterns, and generate alerts. The database schema is straightforward - tables for performance metrics, sessions, trust events, and competency assessments, with appropriate indexes for time-series queries and engineer-specific lookups.

This creates a closed-loop system where both AI and human performance are continuously monitored and maintained. The AI circuit breaker prevents unsafe AI actions. The human performance system prevents unsafe human actions. Together they create defense in depth that makes the entire system trustworthy.

Implementation

We implement this in six phases over eight months.

Months 1-2 are foundation. We build the performance database, implement performance monitoring, add basic fatigue detection, and integrate alerts into Tom Sawyer. Deliverables are a working monitoring system, engineers receiving alerts when fatigue or rushing is detected, and baseline metrics established for all knowledge engineers.

Months 2-3 add trust calibration. We deploy trust monitoring, implement confidence explainability, build the enhanced validation interface with confidence breakdown, and create personal trust calibration dashboards. Deliverables are engineers seeing confidence explanations for all AI suggestions, trust warnings appearing when appropriate, and historical calibration data being tracked.

Months 3-4 add cognitive support. We deploy workload management, implement intelligent task assignment, add break recommendations and enforcement, and enhance Tom Sawyer dashboards with human metrics. Deliverables are workload balanced across the team based on cognitive capacity, automated break recommendations, and real-time cognitive load indicators.

Months 4-6 add competency maintenance. We deploy competency scheduling, implement manual mapping exercises, build knowledge assessment quizzes, and create skill tracking with degradation alerts. Deliverables are periodic competency exercises for all engineers, skill degradation detected and remediated, and training automatically scheduled when needed.

Months 6-7 add organizational safeguards. We deploy production pressure monitoring, build management dashboards for team health, implement safety culture metrics, and add override pattern analysis. Deliverables are management visibility into quality versus pressure tradeoffs, early warning for burnout, and safety culture reinforcement mechanisms.

Months 7-8 are full integration and optimization. We run integration testing across all services, optimize performance, conduct user training and change management, and establish the continuous improvement framework. Deliverables are a fully integrated system, comprehensive training for all users, and an ongoing monitoring and optimization process.

Resource requirements are eight engineers over eight months. Three full-stack engineers for development. One UX designer with human factors focus. One data scientist for trust calibration and performance modeling. Two engineers for integration with existing systems. One QA engineer. One person for training and change management. Total investment around \$1.2 million.

Return on Investment

The cost of not implementing this is substantial. Based on industry data and documented failures of human-AI systems, here's what happens without these safeguards.

Automation complacency causes systematic errors with 80% probability within 12 months. Remediation and data quality impact: \$2-5 million. Trust mis-calibration creates a 15-20% acceptance error rate with 90% probability within six months. Annual rework costs: \$500K-2M. Skill degradation prevents effective intervention with 70% probability within 18 months. External consultant costs per incident: \$200K-500K. Alert fatigue leads to missed critical conflicts with 60% probability within 12 months. Semantic inconsistency remediation: \$1-3M. Cognitive overload requires rework of 30-40% of work with 95% probability within three months. Annual impact: \$300K-800K. Production pressure leads to safety culture erosion with 50% probability within 24 months. Catastrophic failure cost: \$5M-50M+. Mode confusion causes strategic errors on incomplete data with 40% probability within 12 months. Cost per major error: \$500K-2M.

Conservative total over three years: \$10-20 million in preventable costs.

The investment is \$1.2 million over eight months. Three-year value is \$10-20M in prevented costs. ROI is 8x to 17x. Payback period is 4-8 months.

Beyond the numbers, there are intangible benefits. Trustworthy AI governance that can scale. Knowledge engineer retention because you've reduced burnout. Regulatory compliance evidence showing demonstrable human oversight. Competitive advantage from enterprise AI that actually works. Foundation for future AI initiatives that won't repeat the same mistakes.

Success Metrics

We measure success across four dimensions, with specific targets appearing at different phases of implementation.

Human performance metrics show up from month three onward. Validation accuracy stays above 90% across all sessions. Decision quality doesn't degrade over six-hour sessions. Zero critical safety overrides due to production pressure. Engineer burnout indicators stay below 0.3. Session fatigue scores remain below 0.7 throughout the day.

Trust calibration metrics appear from month four. Trust mis-calibration reduces to less than 5% delta from actual accuracy. Over-trust incidents drop 80% from baseline. All engineers access calibration dashboards weekly. Acceptance rate aligns with accuracy within 10%.

System effectiveness metrics come in from month six. The AI Circuit Breaker prevents 100% of high-risk actions. Manual intervention success rate exceeds 95% when the breaker trips. Competency scores stay within 10% of baseline despite automation. Manual mapping exercise scores exceed 80% accuracy. Knowledge assessment pass rate exceeds 85%.

Organizational metrics are visible from month eight. Production pressure scores stay in the normal range below 0.3. Team burnout risk affects less than 10% of members. Safety culture survey scores exceed 4.0 out of 5.0. Zero instances of management pressure to override safety alerts. Workload variance across team stays below 0.3.

Conclusion

Trustworthy engineering isn't complete until it encompasses both the AI and the humans who work with it. We've built trustworthy AI through rigorous technical safeguards - circuit breakers, federated architecture, semantic consistency, explainable reasoning. This framework extends those same principles to create a trustworthy human-AI system.

The difference this makes is straightforward. Without human factors engineering, better AI leads to worse human oversight, which leads to worse overall system reliability. With it, better AI leads to sustained human oversight, which leads to better system reliability.

Week one: AI at 65% accuracy, humans at 95% oversight, system at 95% reliability. Week 52 without framework: AI at 92% accuracy, humans at 70% oversight, system at 85% reliability. Better AI, worse system. Week 52 with framework: AI at 92% accuracy, humans at 94% oversight, system at 99% reliability. Better AI, better system.

This is how trustworthy engineering scales. We don't just make the AI trustworthy. We make the entire system trustworthy by ensuring both components maintain high performance and the interaction between them is well-designed. The AI doesn't degrade because we monitor and maintain it. The humans don't degrade because we monitor and maintain them. The combination is reliable in ways that neither piece alone could achieve.

Every sophisticated automation system that neglected the human dimension has eventually experienced significant failures. Aviation learned this and now trains pilots extensively on automation management. Healthcare is learning it through painful experience with alert fatigue. Industrial control learned it at Three Mile Island and has rebuilt around it. Financial systems learned it through trading algorithm failures.

We have the opportunity to learn from their experience instead of repeating it. The technical architecture we've built is rigorous and well-engineered. This framework completes it by bringing the same engineering discipline to human performance. Together they create semantic governance that's trustworthy not just in principle, but in practice, at scale, over time.

That's the path to AI governance we can actually rely on.